

A EXPERIMENTS

A.1 EXPERIMENTAL SETTINGS

This section details our experimental setup, which is grounded at a substantial scale for academic research. We describe our pretraining and evaluation procedures, justify our learning rate schedule choices, and outline a practical mid-training configuration.

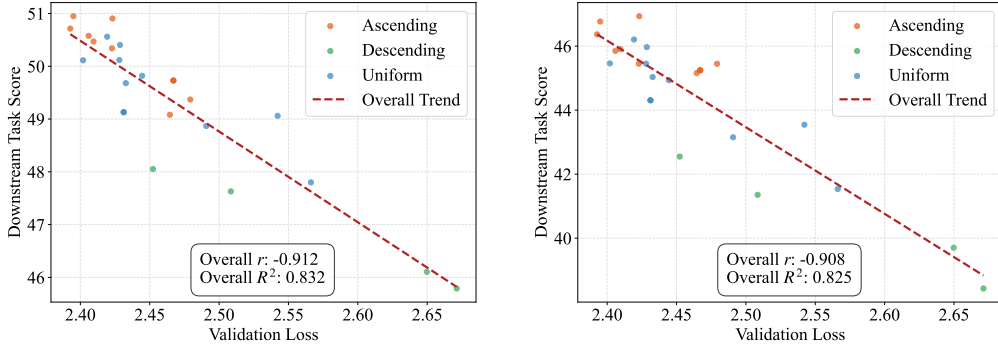
Pretraining Settings. Our experiments are conducted at a substantial scale for academic validation, grounded in the DataComps-LM (DCLM) framework (Li et al., 2024) at the 1B-1x level. We use Qwen2.5-1.5B model (Yang et al., 2024a) for pretraining experiments, which is a modern architecture incorporating SwiGLU activation functions and Grouped-Query Attention (GQA). The training dataset consists of a 30B token subset sample from the first shard of the DCLM-Baseline dataset (Li et al., 2024). For our data curricula, we sort data in ascending order based on DCLM fasttext scores; a reverse curriculum, sorting in descending order, is also used for comparison. After moderate tuning, we set the peak learning rate to 3×10^{-3} , with a sequence length of 4096 and a batch size of 512, which we found provides a good trade-off between throughput and stability. For LR decay schedules, we set the final learning rate to 1×10^{-5} , aligning with optimal settings found in prior work (Li et al., 2024; Luo et al., 2025; Li et al., 2025b). To ensure reproducibility, we provide a detailed list of hyperparameters in Table 3.

Table 3: Model and optimizer hyperparameters for our Qwen2.5-1.5B experiments.

Hyperparameter	Value
<i>Model Configuration</i>	
Sequence Length	4096
Hidden Size	1536
FFN Intermediate Size	8960
Number of Layers	28
Number of Attention Heads	12
Number of Key-Value Heads (GQA)	2
Vocabulary Size	151936
<i>Optimizer Configuration</i>	
Optimizer	AdamW
Weight Decay	0.1
Adam β_1	0.9
Adam β_2	0.95
Adam ϵ	1.0×10^{-8}
Gradient Clipping	1.0

Evaluation Settings. To ensure a robust comparison between methods, we track validation loss during training and evaluate final performance on a comprehensive suite of downstream tasks. Because the data distribution shifts during a curriculum, we created a dedicated high-quality validation set to provide a consistent measure of progress. This set consists of 100,000 of the highest-scoring documents from a partition of the DCLM-Baseline dataset that is disjoint from our training data. For downstream evaluation, we use the OLMES benchmark (Gu et al., 2025a), reporting performance on MMLU (Hendrycks et al., 2021), ARC-easy/challenge (Clark et al., 2018), CommonSenseQA (CSQA) (Talmor et al., 2019), OpenBookQA (Mihaylov et al., 2018), PIQA (Bisk et al., 2020), Social IQA (Sap et al., 2019), and WinoGrande (Sakaguchi et al., 2020), covering world knowledge, common sense, and reasoning capabilities. Among these, we designate MMLU, ARC, and CSQA as *Core* benchmarks, as recent work suggests they have a higher signal-to-noise ratio for distinguishing model performance (Heineman et al., 2025). Furthermore, as shown in Figure 7, the average downstream scores exhibit a strong correlation with validation loss.

LR Schedule Choice Ablation. The decay phase of the WSD schedule can be implemented with various functions. Following prior work (Hägele et al., 2024; Luo et al., 2025), we compared the



(a) Average downstream score over validation loss.

(b) Core downstream score over validation loss.

Figure 7: Downstream task scores and validation losses show high correlation according to the Pearson correlation coefficient (r) and R-square value (R^2). *Average* is the average score of the total 8 downstream tasks and *Core* is the average score of the first 4 downstream tasks (MMLU, ARC-c/e, CSQA) in Tables 1 and 2.

l-sqrt decay function, $\eta(t) = \eta_0 \left(1 - \sqrt{r(t)}\right) + \eta_T \sqrt{r(t)}$, and the *sqrt-cube* function, $\eta(t) = \eta_0 (1 - r(t))^{1.5}$. Both decay functions outperform simpler alternatives like linear decay, and as shown in Table 4, they produce strong and comparable results. We adopt the *l-sqrt* function for our main experiments due to its wide adoption in recent literature (Hägele et al., 2024; Tian et al., 2025). Unless otherwise specified, our experiments use a decay phase ratio of approximately between 15% and 20% of total training steps, consistent with optimal ratios reported in prior work (Hägele et al., 2024; Hu et al., 2024).

Table 4: Models trained under WSD schedules under *l-sqrt* and *sqrt-cube* decay functions produce similar results.

Dataset	Schedule	MMLU	ARC-c	ARC-e	CSQA	OBQA	PIQA	SIQA	Wino.	Avg.
Random	l-sqrt	26.60	27.42	42.28	42.26	37.20	67.85	42.02	51.30	42.12
Filtered	l-sqrt	26.97	30.10	44.04	44.72	36.20	69.15	42.58	51.78	43.19
Random	sqrt-cube	26.62	27.42	42.28	42.42	35.00	68.28	43.65	51.70	42.17
Filtered	sqrt-cube	26.70	31.10	44.04	42.59	36.60	68.44	42.89	52.17	43.07

The Mid-Training Experiment Settings. We model mid-training, an emerging and practical pre-training paradigm (Yang et al., 2025; OLMo et al., 2025; Hu et al., 2024), with a two-phase setup. The initial stable-LR phase uses 29B tokens from the WebOrganizer dataset (Wettig et al., 2025) as lower-quality data, while the subsequent decay phase uses 5B tokens from the higher-quality DCLM-Baseline dataset (Li et al., 2024). The WebOrganizer data has not undergone model-based filtering, whereas the DCLM-Baseline data represents the top 10% of its source, which has a distribution roughly similar to the WebOrganizer data. The LR decays to 1×10^{-5} during the second, high-quality phase.

A.2 PRACTICAL IMPLEMENTATION DETAILS

Weight Computation for WMA. The computation of Weighted Moving Average (WMA) follows Tian et al. (2025), where weights are derived from a hypothetical learning rate schedule. For our experiments, we use a WSD schedule with a *l-sqrt* decay function and a final LR set to 5% of the peak LR. Given normalized LR values $\eta_1, \dots, \eta_N$ at each checkpoint (with $\eta_1 = 1$), the weights are calculated as the drop in learning rate between steps: $w_i = \eta_i - \eta_{i+1}$ for $i < N$, and $w_N = \eta_N$. Suppose $M_0, M_1, \dots, M_N$ are the model parameters of the last $N + 1$ checkpoints, and assume $M_k = M_0 + \sum_{j=1}^k g_j$, where g_j represents the total parameter update between checkpoint $j - 1$ and j . This weighting strategy ensures that the averaged model is equivalent to re-weighting each

update:

$$\sum_{i=1}^N w_i M_i = \sum_{i=1}^{N-1} (\eta_i - \eta_{i+1}) \left(M_0 + \sum_{j=1}^i g_j \right) + \eta_N \left(M_0 + \sum_{j=1}^N g_j \right) = M_0 + \sum_{i=1}^N \eta_i g_i$$

This formulation shows how the weighted average effectively re-weights the parameter updates g_i by their corresponding normalized learning rates η_i , thus simulating the effect of an LR schedule through averaging. Notably, for the *l-sqrt* decay function, this method results in a set of monotonically decreasing weights, as shown in Table 5.

Table 5: Model Checkpoint Weights. Index $-k$ corresponds to the last k -th checkpoint.

Checkpoint Index	-6	-5	-4	-3	-2	-1
Weight	0.4249	0.1760	0.1350	0.1138	0.1003	0.0500

Practical Implementation of CMA (and CDMA). Our Curriculum Model Averaging (CMA) approach consists of a three-stage pipeline, as detailed in Algorithm 1. First, in an offline *data scheduling* stage, the entire training dataset is sorted in ascending order based on a pre-computed quality score. This large-scale sorting is performed efficiently as a one-time process using Apache Spark. A significant practical advantage is that these quality scores (e.g., DCLM fasttext (Li et al., 2024) or PreSelect (SHUM et al., 2025)) are often already generated during data preprocessing for filtering purposes, allowing our method to be integrated into existing pipelines without requiring an additional, costly scoring step. Second, during the *training* stage, we employ a simple warmup-constant LR schedule, forgoing any LR decay. Finally, to ensure the stability of the final parameters, we perform *model averaging* on the last several checkpoints saved during the constant-LR phase. We primarily use Exponential Moving Average (EMA), which assigns higher weights to more recent checkpoints trained on higher-quality data, thereby smoothing parameter variance while emphasizing high-quality signals. For CDMA, the only difference is that instead of entirely forgoing LR decay, we use a moderate decay (e.g., to a final LR of $1/5$ to $1/2$ of the peak LR) and then apply weight averaging. Further exploration of optimal ending LR and the scaling properties of this combined strategy are promising directions for future work.

A.3 ABLATION STUDIES

In this section, we validate the robustness of our findings. We conduct ablation studies to demonstrate that our approach generalizes across a different quality metric (PreSelect) and a different, unfiltered pretraining dataset (WebOrganizer) (SHUM et al., 2025; Wettig et al., 2025).

Ablation on Quality Metric. To test if our approach generalizes to other quality metrics, we conducted experiments using PreSelect scores (SHUM et al., 2025) to order the data. The results in Table 6 show that when using an ascending order of PreSelect scores, both CMA and CDMA outperform the standard WSD baseline. However, we note that ordering by PreSelect scores yielded slightly lower average performance than ordering by DCLM scores in our setup. Moreover, as shown in Figure 8(a), although the ascending-order training shows an overall faster convergence in the latter phase of the training process, the final validation loss goes above the uniform-ordering data training. We conjecture this may be due to an inconsistency, as our base dataset was originally filtered using DCLM scores before being re-sorted by PreSelect scores, while the targeted validation set consists of samples with the highest DCLM scores.

Ablation on Pretraining Dataset. To verify that our method is applicable to broader data distributions, we conducted experiments on the WebOrganizer dataset (Wettig et al., 2025) alone, which has not undergone model-based filtering. As shown in Table 7, combining model averaging with a data curriculum again produces superior results compared to a standard LR decay approach. Notably, in this setting, model averaging without any decay showed a larger benefit than model averaging with moderate decay, possibly indicating that a high ending learning rate is particularly beneficial when high-quality data is extremely sparse.